

CS4243 Cheatsheet

Emilia Ayu

Variables and Operations

Statistics

Mean / Std. dev.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad \sigma = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2}$$

$$\text{Normalisation } x' = \frac{x - \mu}{\sigma} \quad (\text{or min-max: } \frac{x - \min}{\max - \min})$$

$$\text{Gaussian (1D)} \quad G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/2\sigma^2}$$

$$\text{Gaussian (2D)} \quad G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

$$\text{Accuracy: } \frac{TP + TN}{TP + TN + FP + FN} \quad \text{Precision: } \frac{TP}{TP + FP}$$

$$\text{Recall: } \frac{TP}{TP + FN} \quad \text{F1-score: } \frac{2PR}{P + R}$$

Calculus

$$\text{Line eqns: } y = mx + b; x \cos \theta + y \sin \theta = \rho \quad \text{Euclidean dist: } d = \sqrt{\sum_i (x_i - y_i)^2}$$

$$\text{Gradient: } \nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) \quad \text{Laplacian: } \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\text{Orientation: } \theta = \tan^{-1} \left(\frac{f_y}{f_x} \right) \quad \text{Magnitude: } \|\nabla f\| = \sqrt{f_x^2 + f_y^2}$$

Lecture 1 - Computer Vision

Digital Image Acquisition



Illuminance ($i(x, y)$, $0 \leq i \leq \infty$): amount of source illumination incident on scene.

Reflectance ($r(x, y)$, $0 \leq r \leq 1$): amount of source illumination reflected by objects on scene.

Exposure: amount of time incident light can reach imaging sensor.

Aperture: how much light you let in (too much: bokeh, too narrow sharp image)

Shutter Speed: how much time you give light to accumulate (slow: motion blur, fast: freeze action)

	Spatial Resolution	Intensity Resolution
Definition	smallest discernible detail	smallest discernible δ intensity level
Measurement	pixels / unit distance	bit depth / pixel
Determined by	sampling	quantisation
Effects of low res	pixelation	false contouring

File size (bits): $b = M \cdot N \cdot k$

M : rows, N : columns, k : bits/pixel, b : total bits

File Extensions

.bmp: raw bit file (largest size)

.png: predicts pixels based on neighbours, reconstructs original pixel value (lossless)

.jpg: 8x8 image blocks represented by 64 Discrete Cosine Transform coefficients, reconstruction is approximate (lossy). Zigzag ordering captures larger coefficients first; more efficient for Huffman entropy encoding

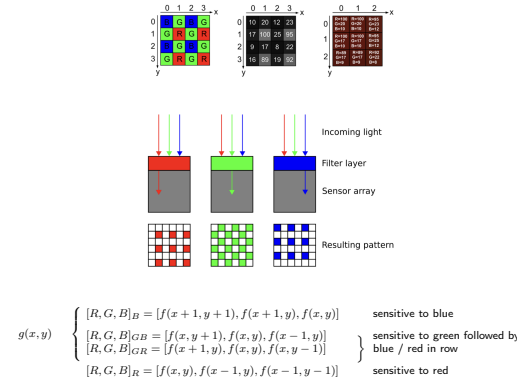
Colour Representation

RGB Space $im(x, y, b) = y$ pixels down, x pixels to right in the b th channel

HSV Space Hue Saturation Value

Bayer Filter

Each pixel only stores one color sample over it (RGB). Reduces intensity resolution.



Lecture 2 - Point Processing and Filtering

Point processing: $x_{ij} = f(p_{ij})$

Filtering: $x_{ij} = f(w_{ij})$

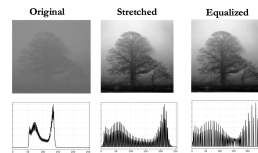
Point Processing

Brightness = $x_{ij} = p_{ij} + b$ (clip when needed)

Intensity scaling = $x_{ij} = a \cdot p_{ij}$

Image Standardisation Preprocess to remove arbitrary brightness/contrast baselines. Algorithms process better when inputs are centered around 0.

Histogram Stretching vs. Equalisation



Stretching: linear rescale to full range

$$s = \frac{r - r_{\min}}{r_{\max} - r_{\min}} (L - 1)$$

Pros: improves contrast; preserves relative brightness relationships

Cons: limited effect if histogram already spans most of the range

Equalisation: redistribute via CDF for uniform histogram

$$s = (L - 1) \sum_{k=0}^r p(k), \quad s_{\text{output}} = k \cdot CDF(s_{\text{input}})$$

Pros: more dramatic contrast change; reveals poorly represented ranges automatically

Cons: can amplify noise; alters clinically irrelevant contrast

Cross-Correlation and Convolution

Purpose: smoothing/averaging with neighbours reduces impulse (salt-and-pepper) noise; also used for

edge detection and template matching.

Cross-Correlation: slide kernel F as-is, multiply & sum

$$x_{ij} = \sum_u \sum_v f_{uv} \cdot p_{i+u, j+v}, \quad X = P \circledast F$$

Convolution: flip kernel 180° first, then multiply & sum. Commutative and associative.

$$x_{ij} = \sum_u \sum_v f_{uv} \cdot p_{i-u, j-v}, \quad X = P * F$$

Equal when: kernel is symmetric under 180° rotation, i.e. $F = F_{\text{flip}}$ (e.g. box filter, Gaussian).

Gaussian Kernel

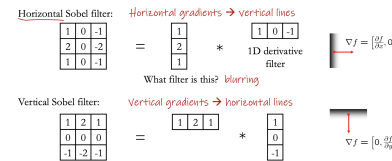
Used as the filter f_{uv} in cross-correlation/convolution above to smooth/blur. Larger σ = more blur.

Detail extraction (unsharp masking):

Details = Original - Blurred $\Rightarrow$ Sharpened = Original + α · Details

Lecture 3 - Gradients

Sobel Filter



Vertical filter S_y detects horizontal edges; horizontal filter S_x detects vertical edges (both across gradient).

The gradient points across the edge, ie direction is normal to the edge. Gradient will point from darker to brighter intensities. The edge occurs at peaks.

A 2D filter is **separable** if it can be written as an outer product of a column vector and a row vector:

$$K = uv^T$$

Convolving with the full 2D kernel K is equivalent to convolving with u then v separately. Cheaper since $1 \times n + n \times 1$ multiplications beat $n \times n$.

Example — Vertical Sobel filter:

1. **Smooth** horizontally: $[1, 2, 1]$
2. **Differentiate** vertically: $[1, 0, -1]^T$

$$S_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Computing gradients

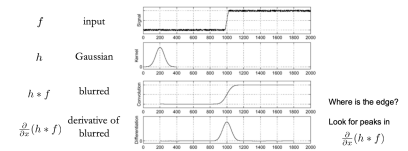
1. Convolve gradient filter with image f to compute image gradient components

$$G_x = \frac{\partial f}{\partial x} = S_x * f \quad G_y = \frac{\partial f}{\partial y} = S_y * f$$

2. Compute gradient orientation and magnitude

Gaussian Derivative and Laplace Filters

Differentiation accentuates noise (multiplicative factor proportional to frequency) $\Rightarrow$ blur before using derivative features to remove noise (but the edge will blur).



Stronger Gaussian returns more high-frequency noise $\rightarrow$ spreads edge across larger region.

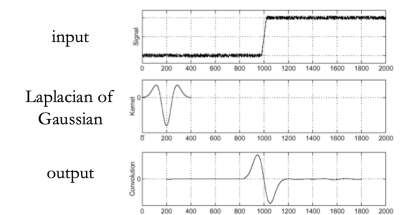
Differentiation Property: $\frac{\partial}{\partial x} (h * f) = \left(\frac{\partial}{\partial x} h \right) * f$

Differentiation is a convolution operation: $f * [1 \ 0 \ -1]$ (x-direction) or $f * [1 \ 0 \ -1]^T$ (y-direction)

Laplacian Operator

$$\text{Continuous: } \Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$
$$\text{Discrete: } \nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

Laplacian of Gaussian Filter



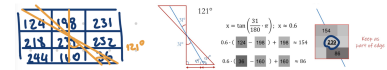
Zero crossings. Localise edges more accurately but less convenient to use.

Canny Edge Detector

Deriv. of Gaussian $\rightarrow$ Grad. mag/orient. $\rightarrow$ NMS $\rightarrow$ Hysteresis thresh.

Non-Maximum Suppression: Only keep local maximums for edges (so edges will be one pixel wide. *Algorithm* Given magnitude map $M(x, y)$ and orientation map $\theta(x, y)$, for every (x, y)

1. Find two neighbours in that direction according to the nearest angle $(+\theta, -\theta)$
2. Interpolate the two neighbour values, with value $= x * (a - b) + b$, where a, b are the two real neighbour pixel magnitudes, and x is the fractional distance, $x = \tan(\theta \text{ mod } 90)$



3. Compare centre pixel to interpolated neighbours.

If $M(x, y) \geq$ both neighbours: keep $M(x, y)$ Else: suppress and set $M(x, y) = 0$

Hysteresis Thresholding: Set two thresholds, H to start edge curves, and L to grow and link edges (if connected to boundary, mark said pixel as boundary)

Algorithm E_H : set of all values passing H , E_L : set of all values passing L . Let $D = E_L - E_H$ (keep only edges of D connected to E_H)

Final image: $E_H \cup D'$

Hough Transform Algorithm Given magnitude map $M(x, y)$ and orientation map $\theta(x, y)$, for every (x, y)

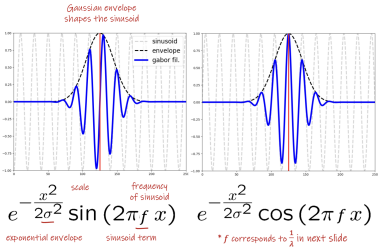
- Quantise parameter space (m, b) by picking a range and step size, then discretise.
- Initialise accumulator array $A(m, b)$. Set each $A(m, b) = 0, \forall m, b$.
- $\forall (x_i, y_i), \forall m : b = x_i m + y_i$. Increment $A(m, b) = A(m, b) + 1$
- Threshold and find local maxima of $A(m, b)$. $y = m^* x + b^*$

Use $x \cos \theta + y \sin \theta = \rho$ — finite parameters $(\theta \in [0, 360])$ vs. unbounded gradient m

Can be made more efficient by voting only near its measured gradient orientation.

Lecture 4 - Regions

Filter banks: collection of multiple filter kernels, where one filter kernel responds to one local pattern. **Gabor Filter:** fix general math function with a range of parameters. Composed of sinusoids and Gaussian envelope.



f : frequency selects spacing between waves; σ : scale determines size of region covered by Gaussian envelope. **2D Gabor formula:**

$$f_{mn} = \underbrace{\frac{1}{2\pi\sigma^2} \exp\left[-\frac{m^2 + \gamma^2 n^2}{2\sigma^2}\right]}_{\text{exponential (Gaussian) envelope selects a local area}} \underbrace{\sin\left[\frac{2\pi(\cos[\omega]m + \sin[\omega]n)}{\lambda} + \phi\right]}_{\text{2D-Sinusoid selects orientation and frequency}}$$

ω – orientation of sinusoid in 2D
 λ – wavelength of sinusoid (inversely proportional to f)
 ϕ – phase of sinusoid; shift from center of kernel
 γ – aspect ratio of envelope coverage ($\gamma = 1$ means circular, otherwise elliptical)
 σ – rate of decay of exponential envelope

Multi-Scale Reasoning

Smoothing suppresses details, which we want to ignore variations in smaller distances to focus on larger structure. *Gaussian image pyramid* → progressively blur then downsample at fixed stride or downsampling rate. We should smooth before downsampling to prevent aliasing (high-frequency can mask as low frequency, ringing).

Scale selection Measure responses on multiple scales. Choose maximum filter size that matches structure size to detect structures.

Single vs Multi-scale Representation Given r_i, ρ_i corresponding to response and scale at i .

Single scale: $r = [r_1, r_2, \dots, r_k]$
Multi-scale: $r = [r_1^{\rho_1}, r_2^{\rho_1}, \dots, r_k^{\rho_1}, r_1^{\rho_2}, r_2^{\rho_2}, \dots, r_k^{\rho_2}]$.

Textures

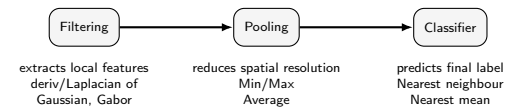
Textures occur over an area. To detect, group pixels by region Ω and average filter responses over region.

- $r(x, y) = [r_1(x, y), r_2(x, y), \dots, r_k(x, y)]$ Run k different filters to get k responses.
- $t_k(\Omega) = \frac{1}{|\Omega|} \sum |r_k(x, y)|$. Take absolute value of every pixel's response to filter k in region Ω and sum. Divide pixels by $|\Omega|$ for average.
 $t_k(\Omega)$ is a pooled response of filter k across Ω
- $t(\Omega) = [t_1(\Omega), t_2(\Omega), \dots, t_k(\Omega)]^T$

To compare textures, $D(\Omega_A, \Omega_B) = ||t(\Omega_A) - t(\Omega_B)||$, i.e. D is the Euclidean distance.

Classifiers

- Nearest Neighbour Classifier: $i^* = \arg \min_i ||t(\Omega) - t(\Omega_i)||$ where $\hat{c} = y_{i^*}$
- Nearest Class Mean: $\mu_c = \frac{1}{N_i} \sum i : y_i = ct(\Omega_c)$ where $\hat{c}(\Omega) = \arg \min ||t(\Omega) - \mu_c||$

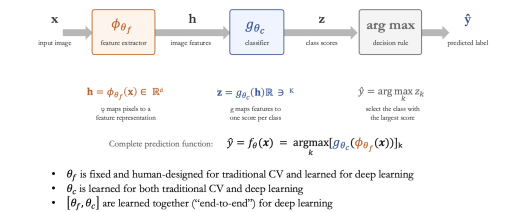


Topic 5 - CNNs

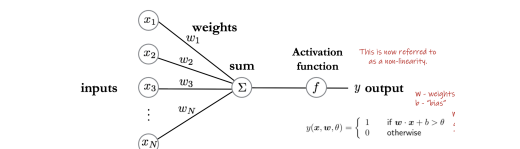
Receptive field: set of image pixels an intermediate output pixel depends on.

Recursive formula (per layer l , with kernel k_l , stride s_l):
 $r_l = r_{l-1} + (k_l - 1)j_{l-1}$, $j_l = j_{l-1}s_l$ base case: $r_0 = 1, j_0 = 1$

Training: Training Images → Image Features → Training with Labels → Learned Classifier
Testing: Test Image → Image Features → Learned Classifier → Prediction
Stages of Classifier



Perceptron



No of learnable parameters = weights (edges) + perceptrons (neurons).

Neurons are total of all possible output nodes (all layers except input). We add perceptrons as bias terms.

MLP Formalism

Input: $\mathbf{a}^{(0)} = \mathbf{u}$; $\mathbf{u}$: input feature vector
Hidden layers:

$$\mathbf{a}^{(\ell)} = \sigma\left(W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \ell = 1, \dots, L - 1$$

$\mathbf{a}^{(\ell)}$: representation in layer ℓ .
 $W^{(\ell)}$: weights in layer ℓ .
 $\mathbf{b}^{(\ell)}$: biases in layer ℓ .
 σ : non-linear activation function
Output layer: $\mathbf{z} = \mathbf{a}^{(L)} = W^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$

Convolution

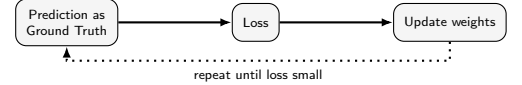
No of learnable parameters:
$$k^2 \cdot C_{in} \cdot C_{out} + C_{out}$$

Strided. Downsample each feature map. Reduce spatial resolution. **Dilated.** Enlarge receptive field without adding params or reducing resolution.

$$H_{output} = \left\lfloor \frac{H_{input} + 2P - D(K - 1) - 1}{S} \right\rfloor + 1$$

Logits: unrestricted real-value scales → convert into probability distribution using softmax.

Loss functions



Examples

- Localisation loss: $L_{loc} = (\hat{x} - x)^2 + (\hat{y} - y)^2$. Distance-based, naturally continuous.
- $L_{0/1}(\hat{y}, y) = \begin{cases} 0, & \hat{y} = y \text{ (correct)} \\ 1, & \hat{y} \neq y \text{ (wrong)} \end{cases}$. Discrete
- Cross-entropy loss: $L_{CE} = -\log(p_y)$. Continuous — softly penalizes low confidence in the correct class

Object Detection

Window-based: Build/train model → Generate candidate windows → Score candidates → Resolve detections with NMS.

Performance Evaluation Area of Overlap Measure
 $AO(B_{gt}, B_p) = \frac{|B_{gt} \cap B_p|}{|B_{gt} \cup B_p|}$

mAP (mean Average Precision): standard object-detection metric, 0–1 or 0–100%. We want high precision (few false positives) and high recall (few false negatives).

- $AP(c, \tau)$ = area under precision–recall curve (class c , $IoU \geq \tau$)
- $mAP(\tau)$ = mean AP across classes

$$AP(c, \tau) = \int_0^1 p(r) dr \quad mAP(\tau) = \frac{1}{|C|} \sum_{c \in C} AP(c, \tau)$$

CNN-Based Object Detection

R-CNNs (Region-Based ConvNets): Generate region candidates, extract CNN features for each region,

regress bounding box (x, y, h, w) for each region, predict label per region.

Fast R-CNNs: Extract CNN features in one pass by locating (x, y) in CNN feature maps and extracting features w ROI pooling.

YOLO (v2) – Anchor-based Detection

- Split image into $S \times S$ cells, each with predefined **anchors** (boxes w/ fixed sizes & aspect ratios)
- Predict at every (grid location, anchor) pair
- Objectness:** $P(obj)$ for box. Predict class + box, skip if no object.
- Class score:** $P(class | obj)$
- Box adjustment:** $P(coords | obj)$, offsets to anchor
- Loss:** $\mathcal{L} = \mathcal{L}_{obj} + \lambda_{box}\mathcal{L}_{box} + \mathcal{L}_{class}$; $\mathcal{L}_{obj}$: all anchors; $\mathcal{L}_{box}, \mathcal{L}_{class}$: object anchors only

Topic 6 - Segmentation

Separation of images into regions/objects.

Semantic: each pixel has a class label. Identify foreground and background.

Instance: Each pixel has an object it belongs to.
Panoptic: Combine semantic and instance.

k-Means Clustering: Iteratively re-assign points to nearest cluster center

- Randomly initialize the cluster centers $c_1, \dots, c_K$
 - For each point p_i , find the closest c_j to put p_i in
 - Set c_j to be mean of points in cluster j
 - Repeat, if c_j have changed up to some threshold
- Pros: Simple, Converges to local min.
 - Cons: Setting K , Sensitive to initial centers (Since k-means converges to local min.), Sensitive to outliers (Can add more clusters), Assumes spherical clusters

Simple Linear Iterative Clustering (SLIC)

- Superpixel:** Group of pixels sharing common traits.
- No of pixels: n_{tp} ; Target no of superpixels: n_{sp}
- Initial width of each superpixel: $s = \sqrt{n_{tp}/n_{sp}}$
- Features: $z = [r, g, b, x, y]$
- Color distance: $d_c = ||\langle r_j, g_j, b_j \rangle - \langle r_i, g_i, b_i \rangle||$
- Spatial distance: $d_s = ||\langle x_j, y_j \rangle - \langle x_i, y_i \rangle||$
- d_{cm} and $d_{sm} = s$ set as max. expected values of d_c and d_s respectively
- $D = \sqrt{(\frac{d_c}{d_{cm}})^2 + (\frac{d_s}{d_{sm}})^2} = \sqrt{d_c^2 + (\frac{d_s}{s})^2} c^2$

- Split img. into grid of size $s \times s$. Set cluster centers as lowest gradient position in 3×3 neighborhood from superpixel center.
 - For each center, check distance to all pixels within $2s \times 2s$ neighborhood. Assign pixels to closest checked center.
 - Update centers using mean; repeat until converged
 - Optional: Replace superpixel region by average value to create stained glass effect
- Modification of k-Means: Not random initialization, Compute pixel's distance only to closest set of centers

Mean-Shift Clustering: Find local density maxima in feature space

- Attraction basin:** Region where all trajectories of centroids lead to same mode
- Cluster:** All data points in attraction basin of a mode

1. For each data point:

(a) Define window around and get centroid

(b) Shift window to centroid

(c) Repeat until window centroid stops moving

• Segment: run mean shift, merge pixels within basin

• Larger window size → Fewer clusters

• Pros: No assumptions on cluster shape, 1 parameter, Finds variable num. of modes (vs. specified k in k-Means), Robust to outliers

• Cons: Choosing h , scales poorly with feature space dimension